동적 계획법

작은 입력의 답을 적어 두고 다음 계산에 다시 씁니다.

동적 계획법

동적 계획법

작은 입력의 답을 적어 두고 다음 계산에 다시 씁니다.

예: 피보나치

0과 1을 적고, 앞의 두 칸을 더해 1,2,3,5를 차례로 적습니다.

6-1 피보나치 수

피보나치 수열은 0, 1로 시작한다. 세 번째 수부터는 바로 앞의 두 수를 더한다.
수의 번호는 0부터 붙인다. 따라서 F(0)=0, F(1)=1, F(2)=1, F(3)=2이다.

정수 n을 받아 F(n)을 반환하는 함수를 작성하시오.

6-1 입력과 반환값

Python: fib_bottomup(n)

n: 구할 수의 번호. 0 이상 90 이하의 정수

반환값

n번째 피보나치 수 하나를 정수로 반환한다. 수열 전체나 계산 횟수를 반환하지 않는다.

선택·판정 조건

0번 값과 1번 값은 각각 0과 1로 정해져 있다.
C에서는 결과가 커질 수 있으므로 long long을 사용한다.

6-1 처음 수 · 두 번째 수

처음 수

n=0
반환값: 0
0번 값은 문제에서 0으로 정했다.

두 번째 수

n=1
반환값: 1
번호를 0부터 세므로 1번 값은 1이다.

6-1 처음 덧셈 · 일반적인 입력

처음 덧셈

n=2
반환값: 1
F(2)=F(1)+F(0)=1+0이다.

일반적인 입력

n=7
반환값: 13
0,1,1,2,3,5,8,13에서 7번 값은 13이다.

6-1 조금 더 큰 입력

조금 더 큰 입력

n=10
반환값: 55
8번 21, 9번 34를 더하면 10번은 55이다.

6-1 예제의 계산 과정

칸 번호 i	계산	적을 값
0	처음 값	0
1	처음 값	1
2	0 + 1	1
3	1 + 1	2
4	1 + 2	3
5	2 + 3	5

같은 방법으로 6번 칸은 8, 7번 칸은 13입니다.

6-1 앞의 두 칸만 읽으면 되는 이유

F(5)를 구할 때 필요한 값은 F(4)=3과 F(3)=2이다. 두 값을 이미 표에 적었다면 다시 구할 필요가 없다. 0번과 1번을 먼저 적고 2번부터 차례대로 채우면, 계산에 필요한 앞의 두 칸이 항상 준비되어 있다.

6-1 코드에서 사용할 값

코드	예: i=5일 때의 뜻
dp[i−1]	바로 앞의 값 dp[4]=3
dp[i−2]	두 칸 앞의 값 dp[3]=2
dp[i]	지금 구할 값 dp[5]=3+2=5
return dp[n]	요청한 번호의 값 하나를 반환

dp는 계산 결과를 적어 두는 배열의 이름입니다.

6-1 Python

```
def fib_bottomup(n):
    if n < 2:
        return n
    dp = [0] * (n+1)
    dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

시간 O(n), 표 공간 O(n)

6-1 C 함수의 인자

함수 원형

long long fib_bottomup(int n)

배열과 작업 공간

반환형은 long long이다.

6-1 C

```
long long fib_bottomup(int n) {
    if (n < 2) return n;
    long long dp[91] = {0};
    dp[1] = 1;
    for (int i=2; i<=n; ++i)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

시간 O(n), 표 공간 O(n)

6-2 인접하지 않은 원소의 최대 합

정수 배열 a에서 원소를 몇 개 고른다. 배열에서 바로 옆에 있는 두 원소는 함께 고를 수 없다. 예를 들어 [1,3,1,5]에서는 3과 5를 함께 고를 수 있지만, 마지막의 1과 5는 함께 고를 수 없다.

조건을 지키는 선택 중 원소의 합이 가장 큰 값을 반환하시오.

6-2 입력과 반환값

Python: nonadjacent(a)

a: 길이 0~10,000인 정수 배열. 각 값은 -1,000,000~1,000,000

반환값

고른 값들의 최대 합을 정수 하나로 반환한다. 고른 위치의 목록을 반환할 필요는 없다.

선택·판정 조건

아무것도 고르지 않는 선택을 허용하며 그 합은 0이다.
이웃 여부는 원래 배열의 위치로 정한다. 배열의 순서를 바꾸지 않는다.
첫 원소와 마지막 원소는 실제로 바로 붙어 있는 경우에만 이웃이다. 배열을 원처럼 연결하지 않는다.

6-2 기본 선택 · 양 끝 선택

기본 선택

a=[1,3,1,5]
반환값: 8
두 번째의 3과 네 번째의 5를 골라 8을 얻는다.

양 끝 선택

a=[5,1,1,5]
반환값: 10
첫째와 넷째 원소는 이웃하지 않는다. 두 5를 함께 고를 수 있다.

6-2 모두 음수 · 빈 배열

모두 음수

a=[-4,-2]
반환값: 0
어느 수를 골라도 합이 줄어든다. 빈 선택의 합 0이 가장 크다.

빈 배열

a=[]
반환값: 0
고를 원소가 없으므로 합은 0이다.

6-2 같은 값 반복

같은 값 반복

a=[2,2,2]
반환값: 4
첫째와 셋째의 2를 고른다. 값이 같아도 위치는 서로 다르다.

6-2 예제의 계산 과정

이번 수	이번 수를 안 고르면	이번 수를 고르면	더 큰 값
1	0	0+1=1	1
3	1	0+3=3	3
1	3	1+1=2	3
5	3	3+5=8	8

마지막 5를 고를 때는 바로 앞의 1을 건너뜁니다.

6-2 마지막 5를 고르면 바로 앞의 1은 빠진다

[1,3,1,5]의 마지막 5를 안 고르면 앞의 세 원소에서 구한 최대 합 3을 그대로 쓴다. 5를 고르면 바로 앞의 1을 빼야 하므로 앞의 두 원소에서 구한 최대 합 3에 5를 더한다. 두 경우를 비교하면 max(3,8)=8이다.

6-2 코드에서 사용할 값

변수	마지막 5를 처리하기 직전
value	이번 수 5
prev1	앞의 세 수 [1,3,1]에서 얻은 최대 합 3
prev2	앞의 두 수 [1,3]에서 얻은 최대 합 3
current	max(3, 3+5)=8

배열을 정렬하면 이웃 관계가 달라집니다. 주어진 순서로 계산합니다.

6-2 Python

```
def nonadjacent(a):
    prev2 = 0
    prev1 = 0
    for value in a:
        skip = prev1
        take = prev2 + value
        current = max(skip, take)
        prev2 = prev1
        prev1 = current
    return prev1
```

시간 O(n), 추가 공간 O(1)

6-2 C 함수의 인자

함수 원형

long long nonadjacent(const int a[], int n)

배열과 작업 공간

a는 값 배열이고 n은 원소 수이다.

6-2 C

```
long long nonadjacent(const int a[], int n) {
    long long prev2=0, prev1=0;
    for (int i=0; i<n; ++i) {
        long long take=prev2+a[i];
        long long current=prev1>take ? prev1:take;
        prev2=prev1;
        prev1=current;
    }
    return prev1;
}
```

시간 O(n), 추가 공간 O(1)

6-3 1로 만드는 최소 연산 횟수

정수 n을 아래 연산으로 바꾸어 1을 만든다. 한 번의 연산에서는 1을 빼거나,
2·3·5 중 하나로 나눈다. 나눗셈은 나누어떨어질 때만 할 수 있다.

n을 1로 만드는 데 필요한 연산 횟수의 최솟값을 반환하시오.

6-3 입력과 반환값

Python: make_one(n)

n: 1 이상 10,000 이하의 정수

반환값

최소 연산 횟수를 정수로 반환한다. 중간에 거친 수의 개수가 아니라 연산한 횟수를 센다.

선택·판정 조건

처음부터 n=1이면 연산하지 않으므로 답은 0이다.
정수 나눗셈으로 나머지를 버리는 연산은 허용하지 않는다. 예를 들어 7을 2로 나누어 3으로 바꿀 수 없다.

6-3 이미 도착 · 한 번에 도착

이미 도착

n=1
반환값: 0
이미 1이므로 연산은 0회이다.

한 번에 도착

n=5
반환값: 1
5÷5=1이다.

6-3 두 번 나누기 · 먼저 1 빼기

두 번 나누기

n=10
반환값: 2
10÷2=5, 5÷5=1로 두 번이다.

먼저 1 빼기

n=26
반환값: 3
26–1=25, 25÷5=5, 5÷5=1로 세 번이다.

6-3 처음에 나눌 수 없음

처음에 나눌 수 없음

n=7
반환값: 3
7–1=6, 6÷3=2, 2÷2=1로 세 번이다.

6-3 예제의 계산 과정

현재 수 i	갈 수 있는 다음 수	각 후보의 횟수	최소
1	이미 1	0	0
2	1	1	1
3	2 또는 1	2 또는 1	1
4	3 또는 2	2 또는 2	2
5	4 또는 1	3 또는 1	1
6	5 또는 3 또는 2	2 또는 2 또는 2	2

다음 수까지 한 번 움직였으므로, 그 수의 답에 1을 더합니다.

6-3 나누어떨어지는 후보만 비교한다

10에서는 9, 5, 2로 갈 수 있다. 9에서 1까지는 최소 두 번, 5와 2에서는 각각 한 번이 필요하다. 여기에 첫 이동 한 번을 더하면 후보는 3,2,2회이고 답은 2회이다. 3으로 나누는 경우는 10이 3의 배수가 아니므로 후보에 넣지 않는다.

6-3 코드에서 사용할 값

코드	x=10에서의 계산
dp[x-1]+1	10에서 9로 이동: dp[9]+1=3
dp[x//2]+1	10에서 5로 이동: dp[5]+1=2
x % 3 == 0	10은 3으로 나누어떨어지지 않으므로 제외
dp[x//5]+1	10에서 2로 이동: dp[2]+1=2

큰 수부터 억지로 규칙을 정하지 않고, 작은 수의 답을 모두 계산해 둡니다.

6-3 Python

```
def make_one(n):
    dp = [0]*(n+1)
    for x in range(2, n+1):
        dp[x] = dp[x-1]+1
        for d in (2,3,5):
            if x % d == 0:
                dp[x] = min(dp[x], dp[x//d]+1)
    return dp[n]
```

시간 O(n), 표 공간 O(n)

6-3 C 함수의 인자

함수 원형

int make_one(int n, int dp[])

배열과 작업 공간

dp는 길이가 n+1인 작업 배열이다. 함수가 dp[1]부터 채운다.

6-3 C

```
int make_one(int n, int dp[]) {
    int divisors[]={2,3,5};
    dp[1]=0;
    for (int x=2; x<=n; ++x) {
        dp[x]=dp[x-1]+1;
        for (int j=0; j<3; ++j) {
            int d=divisors[j];
            if (x%d==0 && dp[x/d]+1<dp[x]) dp[x]=dp[x/d]+1;
        }
    }
    return dp[n];
}
```

시간 O(n), 표 공간 O(n)

6-4 한 번씩만 담는 배낭

무게 제한이 W인 배낭과 여러 물건이 있다. 각 물건에는 무게와 가치가 있다.
물건마다 한 개씩만 있으며, 물건을 쪼개서 담을 수 없다. 담은 물건의 총무게는 W 이하여야 한다.

담을 수 있는 물건들의 가치 합 중 최댓값을 반환하시오.

6-4 입력과 반환값

Python: knapsack2(items, W)

items: 0~100개의 [무게, 가치] 쌍. 무게 1~1000, 가치 0~1,000,000
W: 배낭의 무게 제한. 0~1000의 정수

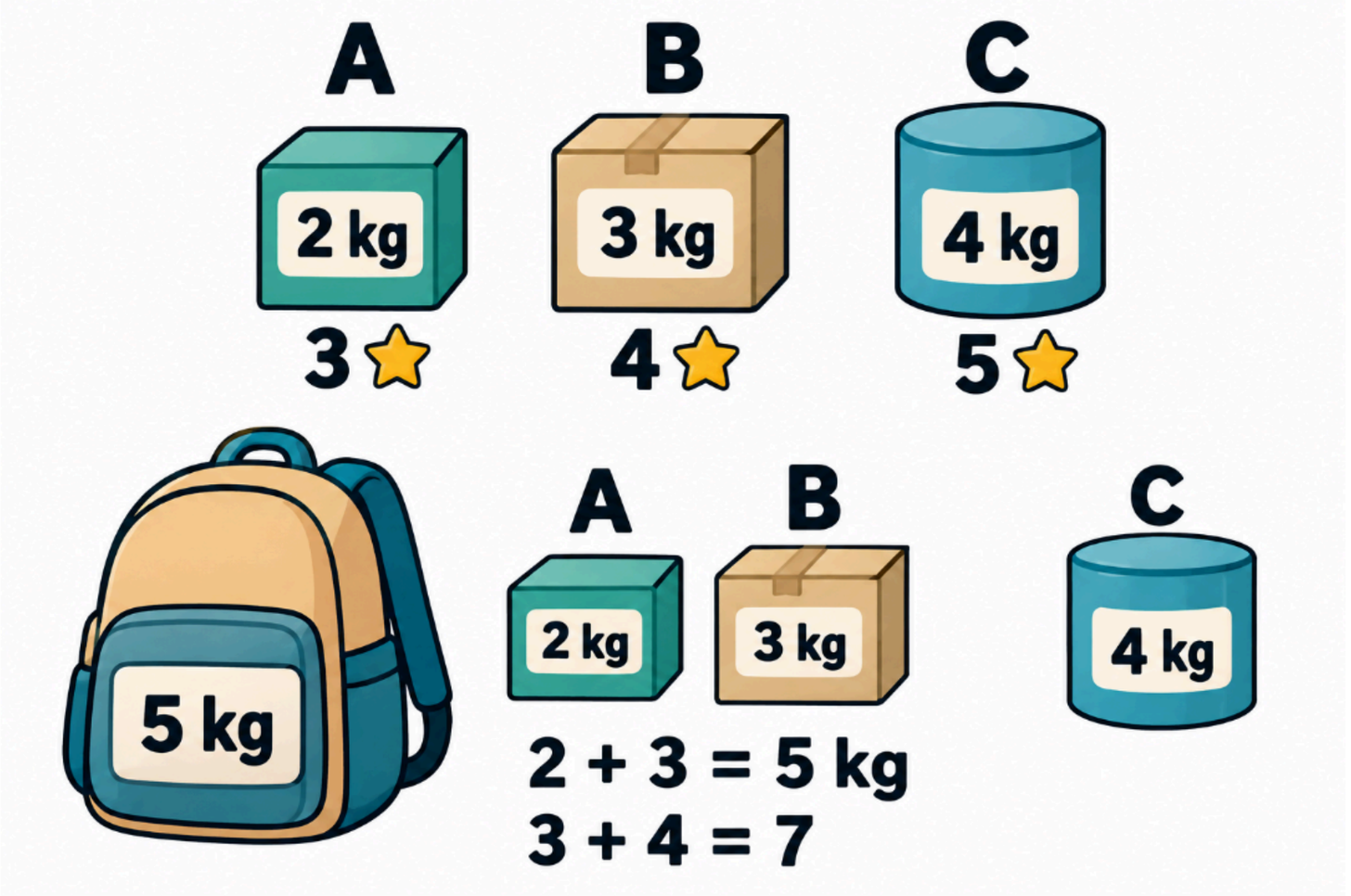
반환값

가치 합의 최댓값을 정수로 반환한다. 물건의 수나 무게 합을 반환하지 않는다.

선택·판정 조건

각 물건은 한 번만 고를 수 있다. 같은 [무게,가치]가 두 번 주어지면 서로 다른 물건 두 개이다.
무게를 정확히 W에 맞출 필요는 없다. W보다 작아도 된다.
아무것도 담지 않는 선택도 허용하며 그 가치는 0이다.

6-4 한 번씩만 담는 배낭



A는 2kg·3점, B는 3kg·4점, C는 4kg·5점이다. 용량 5kg에서 A와 B를 함께 담으면 7점이다.

6-4 두 물건을 함께 담음 · 같은 물건은 한 번만

두 물건을 함께 담음

items=[[2,3],[3,4],[4,5]], W=5
반환값: 7
2kg·3점과 3kg·4점을 담으면 총 5kg·7점이다.

같은 물건은 한 번만

items=[[2,3]], W=4
반환값: 3
용량이 남아도 이 물건은 하나뿐이다. 두 번 담아 6점을 만들 수 없다.

6-4 빈 배낭이 최선 · 정확히 채우지 않아도 됨

빈 배낭이 최선

items=[[6,10]], W=5
반환값: 0
물건이 제한보다 무거워 담을 수 없다.

정확히 채우지 않아도 됨

items=[[2,7]], W=3
반환값: 7
2kg만 담고 1kg를 남겨도 된다.

6-4 물건이 없음

물건이 없음

items=[], W=5
반환값: 0
담을 물건이 없으므로 가치 합은 0이다.

6-4 예제의 계산 과정

사용할 물건	0kg	1kg	2kg	3kg	4kg	5kg
없음	0	0	0	0	0	0
A까지	0	0	3	3	3	3
A,B까지	0	0	3	4	4	7
A,B,C까지	0	0	3	4	5	7

표의 행은 사용할 수 있는 물건, 열은 배낭 용량입니다.

6-4 물건을 넣는 경우와 넣지 않는 경우

2kg·3점인 A와 3kg·4점인 B를 보고 있고 용량이 5kg라고 하자. B를 안 넣으면 A만 담아 3점이다. B를 넣으면 남은 용량은 2kg이고, 그 안에 A를 담아 3+4=7점이다. 이미 처리한 물건까지만 담을 수 있는 윗줄을 읽어야 B를 두 번 쓰지 않는다.

6-4 코드에서 사용할 값

코드	B까지 보고 용량이 5kg인 칸
i=2, c=5	두 번째 물건 B와 용량 5
dp[i−1][c]	B를 안 담음: A만 쓸 때의 3점
dp[i−1][c−w]+v	B를 담음: 남은 2kg의 3점+4점
dp[i][c]	max(3,7)=7점

바로 윗줄을 읽으므로 같은 물건을 다시 담지 않습니다.

6-4 Python

```
def knapsack2(items, W):
    n = len(items)
    dp = []
    for i in range(n+1):
        dp.append([0]*(W+1))
    for i in range(1, n+1):
        w, v = items[i-1]
        for c in range(W+1):
            dp[i][c] = dp[i-1][c]
            if w <= c:
                take = dp[i-1][c-w] + v
                dp[i][c] = max(dp[i][c], take)
    return dp[n][W]
```

물건 수 n, 용량 W: 시간·표 공간 O(nW)

6-4 C 함수의 인자

함수 원형

int knapsack2(const int w[], const int v[], int n, int W, int dp[][1001])

배열과 작업 공간

items를 무게 배열 w와 가치 배열 v로 나누어 받는다. n은 물건 수이다. dp는 (n+1)행, 1001열의 작업 배열이다.

6-4 C

```
int knapsack2(const int w[], const int v[], int n, int W,
    int dp[][1001]) {
    for (int c=0; c<=W; ++c) dp[0][c]=0;
    for (int i=1; i<=n; ++i) {
        for (int c=0; c<=W; ++c) {
            dp[i][c]=dp[i-1][c];
            if (w[i-1]<=c) {
                int take=dp[i-1][c-w[i-1]]+v[i-1];
                if (take>dp[i][c]) dp[i][c]=take;
            }
        }
    }
    return dp[n][W];
}
```

물건 수 n, 용량 W: 시간·표 공간 O(nW)